DOCUMENTATION · RUN IT

Configuration

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=CONFIGURATION. This copy was made on 4 November 2026.

Everything is configured through environment variables (all optional):

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_HOME`	current directory	Folder holding `queryapigate.db` (connections, saved queries, API keys, roles and the audit log).
`QUERYAPIGATE_DATABASE_URL`	unset	A `postgresql://` URL: keep connections, saved queries, run history, API keys, roles and the audit log in that PostgreSQL database instead of `queryapigate.db`, so several instances can share them. Needs `queryapigate[postgres]`; copy an existing store across with `queryapigate migrate-to-postgres`. See the setup guide.
`QUERYAPIGATE_ALLOW_WRITES`	off	Allow `INSERT` / `UPDATE` /DDL. Otherwise only single read-only statements are accepted.
`QUERYAPIGATE_API_KEY`	unset	The shared admin key, with the owner role. When set (or once a scoped key or a named administrator exists), every request except `/health`, `/docs`, `/console`, `/openapi.json` and `/metrics` needs an `X-API-Key` header. Once named owners exist it is the break-glass key: every use is logged and alerted, and it can be removed.
`QUERYAPIGATE_EXPORT_DIRS`	unset	Comma-separated absolute folders a local-folder export destination may write under. Unset: only object-storage destinations (`s3://`, `gs://`, `r2://`) are allowed. Never the server's own home.
`QUERYAPIGATE_MASK_KEY`	unset	The key column masking's `hash` action uses. Unset: one is made once in the store and shared by every instance. Changing it changes every hashed value.
`QUERYAPIGATE_REDIS_URL`	unset	A `redis://` URL: share rate limits and the response cache across every instance and process (`serve`, `mcp`, `events`). Needs `queryapigate[redis]`. Without it, each process counts and caches on its own.
`QUERYAPIGATE_MAX_PAGE_SIZE`	`1000`	Upper limit for `page_size`.
`QUERYAPIGATE_STREAM_MAX_ROWS`	unset	Row cap for a `?stream=true` export. Off (unbounded) by default; a malformed value stops startup.
`QUERYAPIGATE_CORS_ORIGINS`	unset	Websites allowed to call the API from a browser: comma-separated origins such as `https://app.example.com`, or `*`. Off by default.
`QUERYAPIGATE_RATE_LIMIT`	unset	Requests allowed per client address, e.g. `60/minute` (also `second`, `hour`, `day`). Off by default; a malformed value stops startup.
`QUERYAPIGATE_TRUST_PROXY`	`0`	Number of reverse proxies in front of the app whose `X-Forwarded-*` headers are trusted. Set it (usually `1`) behind nginx, a load balancer or a platform router, or every client looks like the proxy.
`QUERYAPIGATE_AUDIT_SYSLOG_URL`	*(unset)*	Stream every audit entry to a SIEM over syslog: `tls://host:6514` or `tcp://host:514`. With `QUERYAPIGATE_AUDIT_SYSLOG_CA`, and `_CERT` / `_KEY` for mutual TLS.
`QUERYAPIGATE_AUDIT_OTLP`	`false`	Stream every audit entry as OTLP log records to the OTLP logs endpoint.
`QUERYAPIGATE_AUDIT_KEY`	*(unset)*	HMAC key (32+ characters) the audit log's hash chain is signed with, so an edit made with only the store can't be re-signed. Several, comma-separated, while rotating it. Unset: plain SHA-256.

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_HSTS_MAX_AGE`	*(unset)*	Seconds browsers should use only HTTPS for this host (`Strict-Transport-Security`), sent only on requests that arrived over HTTPS. Unset: not sent - set it once every client reaches the server over HTTPS.
`QUERYAPIGATE_THREADS`	`8`	Requests one server process serves at once: the Docker image's gunicorn threads (`queryapigate/gunicorn_conf.py`). 1 to 256. The pool size and the `/events` stream cap default from it. See Tune a deployment for throughput.
`QUERYAPIGATE_ACCESS_LOG`	on	The Docker image's gunicorn access log, one line per request on stdout. `off` writes none - for a deployment whose proxy or load balancer logs requests already. `QUERYAPIGATE_LOG_REQUESTS` is QueryAPIGate's own request log, separate from it.
`QUERYAPIGATE_BIND`	`0.0.0.0:5000` in the image	Where the Docker image's gunicorn listens (`queryapigate/gunicorn_conf.py`). Behind the Rust data plane, which then takes port 5000 in a second container sharing the network, `127.0.0.1:5001` (guide 64). Unset outside the image: gunicorn's own default, or its `--bind`.
`QUERYAPIGATE_POOL_SIZE`	one per thread	Idle connections kept per distinct connection setting - by default `QUERYAPIGATE_THREADS` (8). `0` turns pooling off.
`QUERYAPIGATE_POOL_IDLE_TIMEOUT`	`300`	Seconds an idle pooled connection is kept before it is closed.
`QUERYAPIGATE_METADATA_CACHE_TTL`	`0`	Seconds the API key, saved query and connection a request reads from the store are reused, instead of read again each time - see Performance. A change made through this server is seen at once; one made through another instance up to this long later. `0` turns it off.
`QUERYAPIGATE_QUERY_TIMEOUT`	`30`	Seconds a query may run before it is cancelled (HTTP 504). `0` disables the limit. A request can lower it with `?timeout=`, never raise it.
`QUERYAPIGATE_HOST` / `QUERYAPIGATE_PORT`	`127.0.0.1` / `5000`	Bind address for `queryapigate serve`.
`QUERYAPIGATE_DEBUG`	off	Flask debug mode. Never enable on a reachable host.
`QUERYAPIGATE_H2_JAR`	bundled	Path to a different H2 JDBC jar.
`QUERYAPIGATE_SECRET_KEY`	unset	A Fernet key encrypting connection passwords at rest. Off by default (stored as given); needs `queryapigate[encryption]`. A malformed value stops startup.
`QUERYAPIGATE_JSON_LOGS`	off	Emit one JSON object per log line, tagged with the request ID, instead of plain text.
`QUERYAPIGATE_LOG_REQUESTS`	on	A log line for every request, and one for each query it runs (`Executing on ...`). `off` leaves those INFO lines out - errors, warnings, slow queries and agent writes are still logged - for a server whose proxy already logs requests: the lines cost a share of the CPU, and every thread queues on the one handler that writes them. See Performance, v3.
`QUERYAPIGATE_SLOW_QUERY_THRESHOLD`	`1`	Seconds a query may take before it is logged as a warning. `0` disables it.

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_HISTORY_LIMIT`	`50`	Runs kept per saved-query version (and shown in lists).
`QUERYAPIGATE_HISTORY_RETENTION_DAYS`	unset	Keep every run for this many days instead of a per-version count - browse it with `GET /api/v1/history`. Best with `QUERYAPIGATE_DATABASE_URL`.
`QUERYAPIGATE_HISTORY_SAMPLE_RATE`	`1`	Fraction of successful runs recorded; failed runs always are.
`QUERYAPIGATE_HISTORY_ADHOC_LIMIT`	`1000`	Ad-hoc SQL runs (`/execute_sql`, MCP) kept in all, unless a retention period is set.
`QUERYAPIGATE_HISTORY_ADHOC_SQL`	`text`	What an ad-hoc run keeps of its SQL: `text`, `hash` (a SHA-256 only) or `none` - choose `hash` or `none` if SQL may embed sensitive literals.
`QUERYAPIGATE_HISTORY_FLUSH_INTERVAL`	`1`	Seconds between batched history writes; `0` writes inside each request. See Run history.
`QUERYAPIGATE_AUDIT_LOG_LIMIT`	`500`	Administrative-change entries kept in `queryapigate.db`'s audit log; older ones roll off. Always a positive count; a malformed value stops startup.
`QUERYAPIGATE_LOAD_EXAMPLES`	unset	`yes` loads the example APIs (reporting, dashboard, export, partner) at startup - idempotent; a malformed value stops startup. Never removes anything: use `queryapigate examples unload`.
`QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE`	unset	Path to also append every audit entry to, one JSON object per line, never capped - for retention beyond the rolling window above.
`QUERYAPIGATE_JWT_JWKS_URL` / `QUERYAPIGATE_JWT_SECRET`	unset	Accept signed-in users' tokens (`Authorization: Bearer`) from your identity provider or your own backend, with a role (`QUERYAPIGATE_JWT_ROLE` / `QUERYAPIGATE_JWT_ROLE_CLAIM`) for what they may do, and parameters bound to their claims (`from_claim`). See Signed-in users for the related `QUERYAPIGATE_JWT_*` settings.
`QUERYAPIGATE_EVENTS_MAX_STREAMS`	`4`	Concurrent `GET /events` streams the main server holds open (each takes a request thread); beyond it, `503`. `0` turns it off. Unset, never more than half of `QUERYAPIGATE_THREADS`.
`QUERYAPIGATE_EVENTS_PORT`	`5002`	Port of `queryapigate events`, the separate live-events server for many clients - see Live events.
`QUERYAPIGATE_EVENTS_MAX_CONNECTIONS`	`10000`	Open streams one `queryapigate events` process accepts.
`QUERYAPIGATE_EVENTS_POLL_INTERVAL`	`1`	Seconds between `queryapigate events` checks for new runs (on PostgreSQL a notification usually arrives first).
`QUERYAPIGATE_EVENTS_RETENTION_HOURS`	`24`	Hours the event log keeps an event: how far back a `queryapigate events` client can resume with `Last-Event-ID`. A whole number, at least 1.
`QUERYAPIGATE_MCP_PORT`	`5001`	Bind port for `queryapigate mcp` - see documentation/MCP.md.
`QUERYAPIGATE_MCP_MAX_ROWS`	`200`	Row cap for an MCP tool call's result. Always a positive count; a malformed value stops startup.

VARIABLE	DEFAULT	EFFECT
`QUERYAPIGATE_SSO_ISSUER` / `QUERYAPIGATE_SSO_CLIENT_ID` / `QUERYAPIGATE_SSO_CLIENT_SECRET` / `QUERYAPIGATE_SSO_REDIRECT_URL`	unset	Administrators sign in to the Console through your identity provider, their role from their groups (`QUERYAPIGATE_SSO_ROLES`). See Console sign-in through SSO.
`QUERYAPIGATE_MCP_RESOURCE_URL` / `QUERYAPIGATE_MCP_AUTH_SERVER`	unset	Let hosted AI clients sign in through your identity provider (OAuth) - the MCP server's own URL and the provider's issuer, with `QUERYAPIGATE_MCP_ROLE` / `QUERYAPIGATE_MCP_ROLE_CLAIM` and `QUERYAPIGATE_MCP_SCOPE` . See Sign-in for AI clients.

The Console's **Settings** screen shows every one of these with its effective value and whether it was set or is the default (secrets are reported as configured or not, never shown). Settings that hold secrets can also be read from a file - `QUERYAPIGATE_SECRET_KEY_FILE` and the like: see the server's own secrets, from files.

Calling the API from a browser

Browsers refuse cross-origin JSON calls unless the server allows them. List the sites that may call the API:

```shell
QUERYAPIGATE_API_KEY=change-me QUERYAPIGATE_CORS_ORIGINS=https://app.example.com queryapigate serve
```

Preflight checks are answered automatically, and the pagination headers (`X-Has-More` etc.) are exposed to the page's JavaScript. CORS only tells the *browser* which sites may call; it is not authentication, so keep the API key. Avoid `*` without a key: any website a visitor opens could then reach your databases through their browser (the server logs a warning if you start that way).

Rate limiting

`QUERYAPIGATE_RATE_LIMIT=60/minute` gives each client address a bucket of 60 requests that refills steadily, so short bursts work but the sustained rate is capped. Over the limit, requests get `429` with a `Retry-After` header, and every response carries `X-RateLimit-Limit` and `X-RateLimit-Remaining` . The limit is applied before the API key check, so guessing keys is throttled too; `/health` and CORS preflights are never counted. With `QUERYAPIGATE_REDIS_URL` set, limits are counted in Redis - one budget per client address and per key across every instance and process (`serve` , `mcp` , `events`); if Redis fails, each process falls back to counting on its own until it's back, and an alert says so. Without Redis, state is per process: with several workers or instances, the effective limit is multiplied by their number.

An API key can also carry its own `rate_limit` (same grammar), checked in addition to the server-wide limit, never instead of it - a request has to pass both. See Per-key rate limiting.